

FloatLib: Verified Floating-Point Arithmetic in Lean

Robert Joseph George¹

California Institute of Technology, Pasadena, CA, USA

Will Adkisson¹

Washington University in St. Louis, St. Louis, MO, USA

Anima Anandkumar

California Institute of Technology, Pasadena, CA, USA

Abstract

We present FloatLib, a verified arbitrary-precision floating-point arithmetic library in Lean 4 that combines broad format coverage, machine-checked correctness, and efficient certified execution. To our knowledge, FloatLib is the first Lean library to unify IEEE binary and decimal arithmetic, arbitrary-width posits, P3109, and user-defined formats and rounding rules behind interchangeable certified software backends. Every certified backend is proved equal to a complete encoded specification, preserving signed zeros and exceptional values, while numerical theorems connect execution to real rounding, error bounds, and exactness. FloatLib combines exhaustive certified tables for small formats with verified word and limb kernels based on guard-and-sticky invariants and independently checked quotient candidates. Its posit development additionally proves standard rounding thresholds and exact quire accumulation within capacity for arbitrary widths. Across matched workloads, FloatLib achieves speedups of up to $1.46\times$ over FLoPS and $116\times$ over Universal, while remaining slower in some regimes such as binary arithmetic against MPFR. Independent conformance testing includes more than 102 million TestFloat evaluations with zero differences under the tested relation. We release the library, proofs, benchmarks, evaluation data, and guide as open source.

2012 ACM Subject Classification Theory of computation $\rightarrow$ Logic and verification; Mathematics of computing $\rightarrow$ Numerical analysis; Software and its engineering $\rightarrow$ Formal software verification

Keywords and phrases Lean, floating-point arithmetic, IEEE 754, posits, verified software, certified execution

Supplementary Material *Software:* <https://github.com/lean-dojo/FloatLib>

1 Introduction

Large language models now generate programs from problem descriptions [24] and construct formal proofs in Lean [35]. For numerical programs, the statement being proved must account for the arithmetic that executes. A proof of a real-valued identity does not by itself describe an implementation that loses small terms, overflows, or rounds at different points in the computation. These effects cause failures in established software: DeepStability documents numerical bugs in PyTorch and TensorFlow [21]. Verification therefore needs specifications of the arithmetic choices and theorems relating executed operations to their numerical meaning.

These choices are especially visible in mixed-precision machine learning. Storing weights, activations, and gradients in fewer bits reduces memory use and data movement, and can increase arithmetic throughput on hardware that supports those formats [27]. Small gradients can underflow to zero, however, and a nonzero update can be lost when added to a much larger weight. Loss scaling, wider accumulators, and higher-precision weight updates preserve information through these computations [27]. FP8 formats make different choices about exponent range, significand precision, and exceptional values; microscaling adds a shared scale to blocks of values [28, 33]. The dot product shows why the accumulator also belongs

¹ These authors contributed equally to this work.

in the specification. Separate multiplications and additions round twice at each step; a fused multiply-add rounds once [19]. A posit quire can accumulate finite products exactly within its capacity and round only the final sum [34]. These implementations share a real-valued formula but have different rounding errors and exactness properties.

Numerical solvers likewise use several precisions within one algorithm. Iterative refinement can factor a matrix at low precision and compute residuals at higher precision, concentrating expensive arithmetic where it improves the answer. Its error analysis identifies the precision and conditioning assumptions under which the iterations recover an accurate solution [8]. Rigorous interval computation requires each computed interval to enclose the exact value. CoqInterval and ValidSDP use floating-point enclosures and rounding-error bounds in machine-checked mathematical proofs [25]. Such analyses need arithmetic contracts that state their rounding, range, and capacity assumptions explicitly.

Repeated scalar operations also make speed a design constraint: allocation, conversion, and wide intermediate arithmetic recur inside reductions and solver iterations. GMP addresses these costs through algorithms chosen by operand size, machine-word arithmetic, and optimized assembly kernels [15]. MPFR builds on GMP to provide efficient arbitrary-precision arithmetic with correctly rounded results [14]. Verified software must account for these costs while preserving its numerical guarantees. Software implementations also allow experiments with formats that have no dedicated hardware. The cost matters within numerical proofs themselves: the CoqInterval and ValidSDP experiments show how faster arithmetic can reduce whole-proof checking time [25].

To address these requirements, we developed **FloatLib, a verified arbitrary-precision floating-point arithmetic library in Lean 4**. Its scalar contracts cover quantization, mixed-precision operations, and accumulation in verified machine-learning programs, with access to the mathematical analysis available in mathlib [39]. Users can configure precision, exponent widths, bias, and exceptional encodings, or define a representation beyond a radix-and-exponent layout. The interfaces also admit custom rounding rules and shared-scale contexts, covering small quantized formats and wide reference formats within the same library. Encoded specifications describe the computed values; refinement theorems equate implementations with these specifications, and numerical theorems connect the results to real arithmetic. Rounding bounds and exactness results can then be used in loop invariants and accumulation arguments. The custom-format affine proof applies the binary rounding bound, while the quire theorem establishes exactness of the complete accumulation loop.

The mathematical starting point is Flocq’s generic formats, rounding theory, and verified binary arithmetic in Coq [4, 5]. In Lean, FloatSpec develops Flocq-style theory and IEEE models [2], while FLoPS formalizes P3109 semantics and rounding, with executable arithmetic and refinement proofs [9, 10]. FloatLib combines extensible numerical representations with cost-based selection among certified software implementations (Table 1). A rounding theorem can therefore remain independent of the table, word kernel, or limb algorithm selected to implement the operation.

Lean provides a compiled language as well as a proof assistant [12]. Direct evaluation of an exact specification, however, can perform unnecessary work. Exact integers and rationals let a reference operation compute an exact intermediate and round once; a large exponent gap may produce a long integer whose low bits cannot affect the answer, a cost also noted in Flocq [4]. FloatLib retains the exact definitions for reasoning and proves that bounded alignment, truncated intermediates, and specialized storage produce the same encoded results. Small domains admit certified tables; larger formats use word, limb, or exact backends according to their applicability and cost. The refinement proofs must preserve carries and ties while

discarding the low-bit computations that do not affect the result.

We make four contributions:

- **Broad format coverage and arbitrary precision.** FloatLib is, to our knowledge, the first Lean library to combine IEEE binary and decimal, arbitrary-width posits, P3109, and custom formats and rounding rules with interchangeable certified software backends. Shared representation and quantization interfaces also cover fixed point, logarithmic, codebook, and block-scaled representations. Custom binary layouts reuse rounding, error, and exactness theorems (Section 2); distinct contracts cover posit rounding thresholds, exact quire accumulation within capacity, and other numerical families (Section 4).
- **Fast arithmetic with interchangeable backends.** We certify tables for small domains and develop word and limb kernels whose shared rounding invariants justify discarding low bits. Quotient checks separate division correctness from the algorithm that constructs a candidate. A planner selects among certified implementations using their costs and applicability conditions (Section 3).
- **Fully verified software backends.** Every backend admitted by the certified interface has a Lean proof that it returns the encoded specification’s result on every input, including exceptional cases. Operation-specific theorems establish real rounding, error bounds, and exactness under their stated assumptions (Sections 2–4).
- **Comparisons of speed and numerical correctness.** We benchmark standard and custom binary and posit formats against MPFR, Berkeley SoftFloat, and Universal under matching configurations. Further comparisons cover P3109 arithmetic with FLoPS and scalar conversions with TensorLib. Conformance checks include more than 102 million TestFloat evaluations of the binary model; our posit comparisons identify rounding and exponent-packing bugs in the tested SoftPosit generic routines (Section 5).

2 Encoded arithmetic and real rounding

Figure 1 shows the four layers: exact numerical representations, word and limb kernels, format models, and certified execution. Let W_F denote the encoded values of a format F , and S_F° its reference operation for $\circ$. Implementation theorems identify kernels with S_F° ; numerical theorems describe the mathematical interpretation of the result.

2.1 Representations and rounding contracts

A format’s storage and meaning have separate interfaces in `FloatLib.Numerics` (namespace and setup declarations omitted):

```
class EncodedFormat (F : Type u) where
  Code : Type v
  Scalar : Type w

class FormatSemantics (F : Type u) [EncodedFormat F] where
  denote : EncodedFormat.Code (F := F) →
    NumericalValue (EncodedFormat.Scalar (F := F))
```

`NumericalValue` distinguishes finite scalars, infinities, and exceptional values. Neither interface assumes a radix or bit layout. A quantization contract relates an exact input to an output in a given context; the context can carry a rounding mode, a shared scale, or random bits. A new family supplies its denotation, operations, and proofs of these contracts. Existing families already supply them for layouts satisfying their parameter conditions.

A nonuniform four-entry codebook illustrates how a new representation reuses the library’s nearest-entry quantizer:

■ **Table 1** Format scope, execution, and proof contracts of related systems. Custom formats include both new layouts and new representations or rounding rules; Section 6 explains the distinctions.

Work	Formats and extensibility	Execution	Connection to proofs
Arithmetic libraries and interfaces			
Flocq [4, 6]	Generic radix and exponent function; arbitrary-precision binary	Rounding and binary arithmetic	Generic theory and verified operators
FloatSpec [2]	Generic formats; IEEE binary	Exact operators; noncomputable rounded IEEE models	Real-value and Hoare-style specifications; native-model bridges
FLoPS [9, 10]	Parameterized P3109; rounding policies	Executable P3109 arithmetic	Refinement and stochastic error bounds
Primitive Floats [3, 25]	Binary64; interval computation	Native floating-point primitives	Primitive axioms connected to Flocq
TensorLib [38]	Low-precision tensor dtypes	Specialized casts; native arithmetic	Tensor interfaces over native scalars
FloatLib (ours)	Arbitrary precision; binary, decimal, posits, P3109; custom formats and rounding	Certified table, word, and limb kernels; cost-based selection	Encoded refinement, numerical bridges, custom contracts
Reasoning about expressions and programs			
Gappa [11]	Exact and rounded expressions	Interval and error analysis	Generated proof certificates
VCFloat2 [1]	User-defined formats and operations	Reflective roundoff analysis	Error bounds for Coq and VST proofs
SymFPU [7]	IEEE/SMT floating point	Concrete or symbolic bit vectors	Floating-point encodings for SMT reasoning

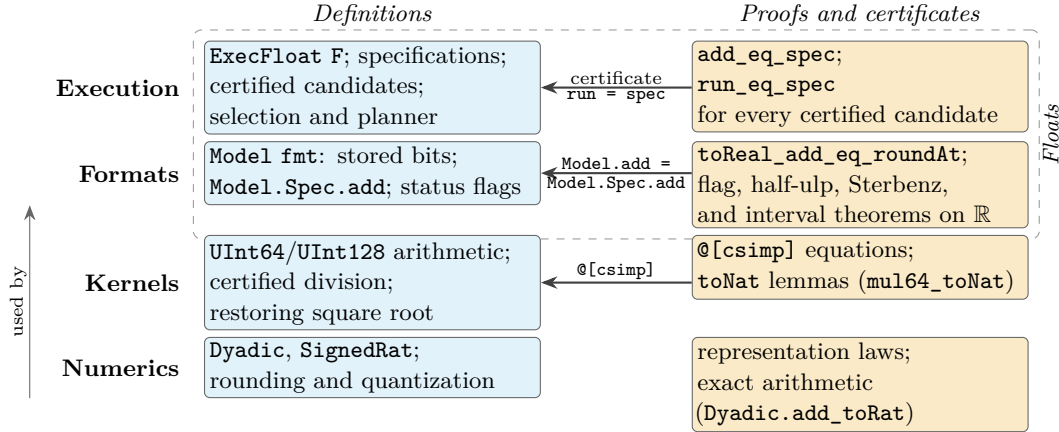
```
import FloatLib
open FloatLib.Floats

def palette : Formats.Codebook 2 ℚ where
  denote w := .finite (match w.toNat with
    | 0 => -1 | 1 => 0 | 2 => 1 / 3 | _ => 2)
abbrev C := ExecFloat.Codebook palette
example : palette.nearestCode (1 / 2) = some 2 := by decide +kernel
```

The result is code 2, denoting $1/3$. The generic `nearestCode_spec` theorem proves that the selected finite entry minimizes distance to the input; ties select the lower code. Users can replace that rule and prove a different quantization contract over the same table. The carrier `C` retains the table in its type, so equal-width codebooks cannot be silently interchanged.

2.2 Binary representations

A binary descriptor records an exponent-field width e_w , a fraction-field width f_w , a bias b , and an exceptional-encoding policy. Its stored values are bit vectors of width $1 + e_w + f_w$. The descriptor’s proof fields require $e_w \geq 2$, $f_w > 0$, and a bias that can encode the finite normal number one. The same descriptor covers standard and custom layouts. Binary32 instantiates $(e_w, f_w) = (8, 23)$; the custom 24-bit layout $(8, 15)$ has precision 16 and uses the same definitions and theorems.



■ **Figure 1** FloatLib’s four functional layers. Definitions (blue) appear beside their proofs and certificates (yellow), including proof fields erased at compilation. The upward arrow summarizes dependencies from Numerics and Kernels into Floats; Formats and Execution also import one another. Horizontal arrows show equations used to certify execution or replace a definition during compilation.

Decoding produces a finite dyadic, an infinity, or a NaN. A finite dyadic has the form $(-1)^s m 2^q$, with natural significand m , integer exponent q , and a Boolean sign s . Retaining the sign at $m = 0$ distinguishes the two zeros. The reference operations handle exceptional cases according to the descriptor, compute finite intermediates exactly, and round and pack the result. For fused multiply-add, the intermediate is the exact dyadic $xy + z$, with one final rounding. Equality in W_F preserves the specification’s choice of zero sign and exceptional encoding. Status-returning operations additionally expose five flags: invalid, division by zero, overflow, underflow, and inexact. These are data returned with the result, not an implicit state of the host processor. The binary status layer detects tininess after rounding to the destination precision with an unbounded exponent; underflow also requires inexactness.

The custom layout illustrates why the rounding point matters under nearest-even rounding. For $a = 1 + 2^{-9}$, $x = 1 - 2^{-9}$, and $b = -1$, all three operands are exact and fused multiply-add returns -2^{-18} . Separate multiplication rounds $1 - 2^{-18}$ to 1, since the spacing immediately below one is 2^{-16} ; adding b then returns zero. Both computations represent the real expression $ax + b$, but they implement different operations.

The real interpretation, written val_F , forgets the sign of zero. Its Lean definition is `total` and assigns zero to nonfinite values. Numerical theorems using this interpretation therefore state finiteness explicitly. For directed bounds that may overflow, the extended reals provide a more suitable codomain.

2.3 Generic rounding theory

The mathematical theory follows Flocq’s radix-and-exponent formulation [4, 5]. A radix $\beta \geq 2$ and an exponent function $\varphi : \mathbb{Z} \rightarrow \mathbb{Z}$ define a representable grid. For $x \neq 0$, choose e so that $\beta^{e-1} \leq |x| < \beta^e$ and set $c(x) = \varphi(e)$. The value x is representable when $x\beta^{-c(x)}$ is an integer. Fixed point takes $\varphi(e) = e_{\min}$; an unbounded precision- P format takes $\varphi(e) = e - P$; gradual underflow takes

$$\varphi(e) = \max(e - P, e_{\min}). \quad (1)$$

Validity conditions on φ make the local grids compatible across magnitude intervals, with additional monotonicity hypotheses where needed. Rounding reduces to an integer decision on the scaled significand:

$$R(x) = \text{rnd}\left(x\beta^{-c(x)}\right)\beta^{c(x)}. \quad (2)$$

Nearest-even uses nearest-integer rounding with an even integer chosen at a tie. Directed rounding replaces that integer operation by floor, ceiling, or truncation. The generic theory proves representability, order properties, and exactness on representable inputs. Its real-valued statements use `mathlib` [39], so the rounding results can be combined with its analysis library.

For a binary descriptor, set $P = f_w + 1$, $e_{\min} = 1 - b - f_w$, and $\beta = 2$ in (1). Write R_F for nearest-even rounding on this grid, with $R_F(0) = 0$, and set $\text{ulp}_F(x) = 2^{c(x)}$ for $x \neq 0$ and $\text{ulp}_F(0) = 2^{e_{\min}}$. The grid models gradual underflow but has no upper exponent bound. It is therefore meaningful to round every real number even when a corresponding finite word cannot be stored.

► **Theorem 1 (Rounding error).** *For every real x , $|R_F(x) - x| \leq \frac{1}{2} \text{ulp}_F(x)$. If $x \neq 0$ and $|x| \geq 2^{1-b}$, then $|R_F(x) - x|/|x| \leq 2^{-P}$.*

Proof. Nearest-integer rounding changes the scaled significand by at most $1/2$; multiplying by the positive grid unit gives the absolute bound. In the normal range, $c(x) = e - P$ and $|x| \geq 2^{e-1}$ give the relative bound. Below it, constant grid spacing preserves the absolute bound but not a uniform relative bound. ◀

2.4 Real interpretation of encoded operations

The binary bridge applies to descriptors with IEEE exceptional encodings and the conventional bias $2^{e_w-1} - 1$. This is the library predicate `isIEEE`, which admits user-defined widths with these conventions without requiring a standardized layout. The bridge theorem identifies packing an exact dyadic with rounding its real value:

$$\text{val}_F(\text{roundDyadic}_F(d)) = R_F(d_{\mathbb{R}}), \quad (3)$$

provided the packed word is finite. The finiteness condition is essential: the grid rounder has no overflow, whereas an encoded operation may return infinity. The proof compares the integer quotient and remainder used by the two rounders: normal packing, subnormal packing, and a carry into the next exponent preserve the same nearest-even decision. A range argument rules out exceptional packing under the finiteness premise.

► **Theorem 2 (Finite arithmetic bridge).** *Let F satisfy the preceding IEEE conventions. If x, y and their reference sum are finite, then*

$$\text{val}_F(S_F^+(x, y)) = R_F(\text{val}_F(x) + \text{val}_F(y)).$$

The analogous equations hold for subtraction and multiplication when their operands and respective results are finite. For fused multiply-add with finite operands and result,

$$\text{val}_F(S_F^{\text{fma}}(x, y, z)) = R_F(\text{val}_F(x) \text{val}_F(y) + \text{val}_F(z)).$$

Proof. The reference operation first computes its finite dyadic intermediate exactly. Its real interpretation is the corresponding real sum, difference, product, or product-and-sum. Applying (3) supplies the single final rounding. ◀

Combining Theorems 1 and 2 bounds the error by half an ulp of the exact result. In Section 3.5, a Lean example applies these results to the custom affine operation without unfolding packing or the selected backend.

Division has the same nearest-even connection when the divisor is nonzero and the operands and result are finite. Square root requires a finite operand that is nonnegative or a zero of either sign; its proof also establishes result finiteness. Directed addition, subtraction, multiplication, division, and square root instead provide outward enclosures in the extended reals under their respective domain conditions. Infinities can then be valid endpoints after overflow. These statements concern the named arithmetic operations; elementary function contracts are discussed in Section 4.2.

2.5 Exact subtraction with gradual underflow

Sterbenz’s lemma states that subtracting nearby floating-point values is exact [37, 30].

► **Theorem 3** (Encoded Sterbenz lemma). *Let x, y be finite words in an IEEE-style descriptor, with positive values $u = \text{val}_F(x)$ and $v = \text{val}_F(y)$. If $u \leq 2v$ and $v \leq 2u$, then $\text{val}_F(S_F^-(x, y)) = u - v$. No separate finiteness assumption on the result is needed.*

Proof. Both operands, and hence their difference, are integer multiples of $2^{e_{\min}}$. If $|u - v| \leq 2^{e_{\min} + P - 1}$, this lattice property places the difference in the gradual-underflow grid. For larger differences, embed the operands in the unbounded precision- P format, apply its Sterbenz theorem, and use the magnitude bound to return to the destination grid. Rounding therefore leaves the difference unchanged. Since $|u - v| \leq \max(u, v)$ also bounds it by a finite input, subtraction cannot overflow; the arithmetic bridge applies with this derived finiteness proof. ◀

Cancellation can produce a subnormal even from normal operands, so replacing gradual underflow by flush-to-zero would invalidate the conclusion. The theorem concerns the real value of the difference; the encoded operation also determines the sign of an exact zero.

3 Efficient execution by refinement

Backend refinement proves that a cheaper implementation returns the same encoded result. This equality lets the planner choose a kernel while preserving the numerical theorems used to reason about the program.

3.1 Execution certificates and backend selection

The common interface, `ExecFloat.Backend.Certified`, pairs an implementation with a proof of equality to its specification (comments and setup declarations omitted):

```
structure Certified {α : Type u} (spec : α) where
  estimate : Candidate
  run : α
  run_eq_spec : run = spec
```

The type α describes the whole operation, whether binary arithmetic, a unary function, FMA, or an accumulator update. The function equality `run_eq_spec` covers every operand, including exceptional values; `Candidate` contains the cost estimates used for selection.

A codec connects a family’s chosen carrier, such as a machine word, to its encoded model and proves the round-trip laws. Lifting a model operation through this codec gives the specification on the carrier; backend certificates prove equality with that lifted operation.

The planner folds over certified alternatives to a mandatory certified baseline, comparing estimated warm and cold costs under a user-selected policy and applying memory ceilings to replacements. Its result still has type `Certified spec`: the selection theorem is `run_eq_spec`, regardless of the cost model’s accuracy. The baseline remains available even if it exceeds the ceilings, so these are selection preferences rather than proved resource bounds.

Kernel eligibility depends on descriptor fields and the operation. For example, a binary two-word kernel requires the IEEE-style conventions, more than 64 stored fraction bits, and at most 128 encoded bits. These conditions permit a proof over all layouts that fit the representation. An eligible kernel can still decline an operand outside its specialized case; its complete dispatcher then calls the reference implementation. The equality theorem covers both branches. Direct entry points and inlining let closed configurations avoid constructing a planning record at each call, while the same certificate identifies their meaning.

3.2 Tables, words, and limbs

A binary operation on an eight-bit domain has only 2^{16} input pairs: a complete table replaces decoding, arithmetic, and rounding by indexing, with each entry proved equal to the reference result. For larger layouts, word kernels keep significands and products in bounded unsigned words with explicit carries and borrows. At wider precisions, limb arrays avoid converting every intermediate to the reference representation.

Aligning $a2^d$ and b exactly can construct an integer whose length grows with the exponent separation d . If cancellation cannot expose the omitted bits, a short retained prefix and one sticky bit suffice for the final rounding. A cancellation guard justifies computing with this summary. FMA reuses the argument after forming an exact product, preserving its single rounding.

Division instead separates proposal from acceptance: a bounded multiplication and remainder comparison check a proposed quotient, and a proved divider handles rejection (Figure 3). We measure the complete operations on a common scalar workload (Section 5); the timings do not isolate the contribution of each optimization.

3.3 Guard and sticky invariants

A kernel often needs only a short prefix of an exact intermediate. The problem is to summarize the discarded suffix without changing a later rounding decision. For a natural number x and a shift j , define

$$J_j(x) = \begin{cases} \lfloor x/2^j \rfloor, & x \bmod 2^j = 0, \\ \lfloor x/2^j \rfloor \text{ OR } 1, & x \bmod 2^j \neq 0. \end{cases}$$

This is *shift right with jamming*: a nonzero discarded suffix sets the low retained bit. Each word or limb implementation is proved equal to this representation-independent definition.

To see what must be preserved, write, for $s > 0$,

$$x = q2^s + g2^{s-1} + r, \quad g \in \{0, 1\}, \quad 0 \leq r < 2^{s-1}.$$

Rounding $x/2^s$ to nearest-even gives

$$E_s(x) = q + 1[g = 1 \wedge (r \neq 0 \vee q \text{ odd})]. \quad (4)$$

The retained quotient, guard bit g , and nonzeroness of the remaining suffix determine the answer. Figure 2(a) shows which information survives the shorter representation.

► **Theorem 4** (Rounding after jamming). *For $x, j, s \in \mathbb{N}$ with $j + 2 \leq s$, $E_{s-j}(J_j(x)) = E_s(x)$.*

Proof. For $k > 0$, setting the low bit cannot affect the quotient above position k or the bit at position k . The shared integer lemmas establish

$$\begin{aligned} \lfloor J_j(x)/2^k \rfloor &= \lfloor x/2^{j+k} \rfloor, \\ \text{bit}(J_j(x), k) &= \text{bit}(x, j+k), \\ J_j(x) \bmod 2^k \neq 0 &\iff x \bmod 2^{j+k} \neq 0. \end{aligned}$$

The last line is the suffix invariant. If the initial shift was exact, it follows from divisibility. Otherwise the jammed word is odd and the original discarded suffix is nonzero. Apply these identities at the retained quotient and guard positions, and substitute into (4). The inequality $j + 2 \leq s$ places the jam strictly below the guard. ◀

The separation condition cannot be weakened in general. For $x = 5$, $j = 1$, and $s = 2$, jamming gives $J_1(5) = 3$. Rounding $3/2$ to even gives 2, whereas rounding $5/4$ gives 1. Here the jam has replaced the guard bit.

The wide-limb subtraction kernel gives a concrete use of the theorem. It stores 32-bit limbs and admits IEEE-style descriptors wider than 128 bits with exponent fields of at most 32 bits. Let p be the stored fraction width, $a, b \geq 2^p$ the magnitudes to be subtracted, and $d = s_a - s_b \geq 0$ their scale difference. The exact aligned integer is $x = a2^d - b$. Writing $\ell_a = \lfloor \log_2 a \rfloor$ and $\ell_b = \lfloor \log_2 b \rfloor$, the compressed branch checks $d \geq 3$ and $\ell_b + 2 \leq \ell_a + d$, then sets $j = d - 3$. This guard bounds cancellation: $b < 2^{\ell_a + d - 1}$ while $a2^d \geq 2^{\ell_a + d}$. Consequently $x \geq 2^{p+j+2}$. For the final rounding shift $s = \lfloor \log_2 x \rfloor - p$, this supplies exactly $j + 2 \leq s$. The limb calculation must also construct $J_j(x)$ correctly. Decompose $b = Q2^j + r$, with $0 \leq r < 2^j$. When $r \neq 0$,

$$x = (8a - Q - 1)2^j + (2^j - r), \quad J_j(x) = (8a - Q - 1) \text{ OR } 1.$$

The final one in the subtraction is the borrow shown in Figure 2(b). When $r = 0$, the result is simply $8a - Q$. The guard also proves that the limb subtraction cannot underflow. The representation lemmas identify these limb operations with $J_j(x)$; Theorem 4 then identifies their rounding with rounding the exact x .

The complete subtraction theorem also covers exact alignment and reference fallbacks outside the compressed or normal-result branches. Decoding gives model subtraction; the codec inverse supplies the encoded equality required by **Certified**. Wide-limb FMA first forms an exact limb product, then applies the same alignment theorem to the product and addend. Because the theorem needs no upper significand bound, it handles the longer product without intermediate rounding. Appendix A gives the complete dispatcher argument.

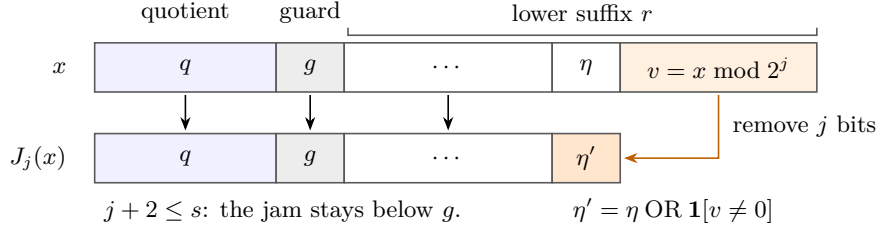
3.4 Certified quotient checking

The divider checks a proposed answer independently of the algorithm that found it. Given normalized significands n, d with $p + 1$ bits, where $64 < p \leq 126$, the two-word backend scales the numerator by 2^s , choosing $s = p$ if $n \geq d$ and $s = p + 1$ otherwise. A radix-2³² implementation of Algorithm D [22] proposes a quotient q and remainder r . The backend accepts them only after independently checking

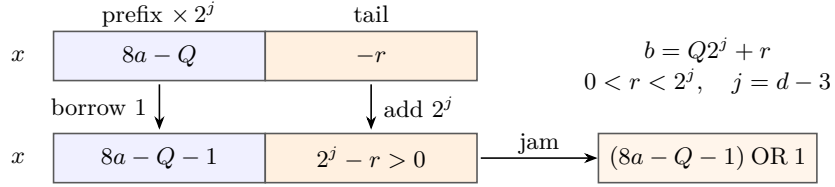
$$qd + r = n2^s, \quad r < d. \tag{5}$$

These relations characterize Euclidean division, so any representable pair passing the check suffices for the accepting branch. The check and fallback supply the same relation to the final rounder (Figure 3).

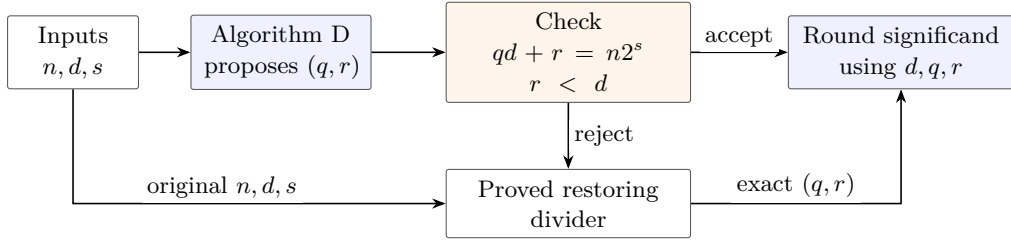
(a) A sticky bit preserves the rounding decision



(b) Account for the subtraction borrow before jamming



■ **Figure 2** Information preserved by compressed alignment. (a) The bit η is at position j before the shift and position zero afterwards. Jamming summarizes the removed tail without changing q , g , or whether the lower suffix is zero; groups are schematic and may be empty. (b) In the guarded subtraction $x = a2^d - b$, a nonzero remainder borrows from the prefix. Its complemented tail is still nonzero, so the low bit is then set. If $r = 0$, the prefix is simply $8a - Q$.



Both routes use the same rounding rule: compare $2r$ with d , then resolve ties by parity.

■ **Figure 3** Checked division for $2^p \leq n, d < 2^{p+1}$, $64 < p \leq 126$, and $s = p + 1[n < d]$. The accepting branch uses the checked equation and remainder bound. The fallback establishes them by a restoring-division invariant and needs no second check. The Boolean check runs during execution; its soundness proof is erased.

The runtime check widens a two-word product to four words, adds the remainder, rejects a final carry, and checks equality with the shifted numerator and the bound $r < d$. The zero-carry conjunct lets the proof interpret this as an exact natural-number equation. The 128-bit argument bounds already give $qd + r < 2^{256}$, so the carry test excludes no representable pair. The precision bound also places $n2^s$ within four words.

► **Theorem 5 (Soundness of the quotient check).** *Let $64 < p \leq 126$, $n, d, q, r < 2^{128}$, and $s \in \{p, p+1\}$. If the native certificate accepts (q, r) for the shifted numerator $n2^s$ and divisor d , then*

$$q = \lfloor n2^s / d \rfloor, \quad r = (n2^s) \bmod d.$$

Proof. The multiplication and addition refinement theorems recover the exact natural-number equation from the checked word operations and zero carry. The comparison theorem

gives $r < d$, hence $d > 0$. Uniqueness of Euclidean division then identifies both quotient and remainder. $\blacktriangleleft$

A rejected candidate falls back to a proved restoring divider satisfying the same equation and bound. One correctness theorem therefore covers both branches. The Boolean check executes at runtime; its soundness proof is erased.

Once the pair is exact, rounding compares $2r$ with d and increments q when $2r > d$, or when $2r = d$ and q is odd. Capacity hypotheses prevent overflow when doubling the remainder or incrementing the quotient. The native rounding theorem identifies the result with exact nearest-even quotient rounding, after which exponent handling and packing complete the backend refinement.

3.5 A rounding bound for a custom format

For the custom 24-bit format of Section 2, we prove both encoded equality and a real-valued rounding bound for $ax + b$:

```
import FloatLib
open FloatLib.Floats Formats.BinaryInterchange
open ExecFloat.Binary (toModel isFinite)

abbrev F := ExecFloat.Binary 8 15
noncomputable abbrev val (x : F) : ℝ := Model.toReal (toModel x)
def affine (a x b : F) : F := ExecFloat.fma a x b

theorem affine_encoded (a x b : F) :
  affine a x b = ExecFloat.Spec.fma a x b :=
  ExecFloat.Proof.fma_eq_spec a x b
```

The encoded equation holds for every operand. Decoding it through the codec gives a one-rounding theorem for finite inputs and output:

```
theorem affine_rounds (a x b : F)
  (ha : isFinite a = true) (hx : isFinite x = true)
  (hb : isFinite b = true) (hout : isFinite (affine a x b) = true) :
  val (affine a x b) =
    Model.roundAt (ExecFloat.Binary.format 8 15) (val a * val x + val b) := by
  have hmodel : toModel (affine a x b) =
    Model.fma (toModel a) (toModel x) (toModel b) := by
    rw [affine_encoded, Model.Proof.fma_eq_spec]
    exact ExecFloat.ModelCodec.decode_liftTernary _ a x b
  simpa only [val, hmodel] using
    Model.toReal_fma_eq_roundAt _ _ _ (by decide) ha hx hb (hmodel ▷ hout)
```

The proof applies Theorem 2; `decide` discharges the closed format condition. Its hypotheses allow subnormal inputs and need no separate finiteness assumption on the product. For the exact real expression $z = \text{val}_F(a) \text{val}_F(x) + \text{val}_F(b)$, Theorem 1 then gives $|\text{val}_F(\text{affine}(a, x, b)) - z| \leq \frac{1}{2} \text{ulp}_F(z)$.

3.6 Parsing and Lean's native types

The configured binary types parse decimal, hexadecimal, dyadic, and special-value text through the same descriptor model. `parse` defaults to nearest-even, with options for rounding direction, exception status, and input limits. We prove that successful bounded parses agree with the unbounded parser and that changing the storage carrier preserves the result and status. Formatting provides exact decimal and hexadecimal output, or fixed and scientific decimal output at a requested precision. For finite values, the latter two have proved nearest-even error bounds on the value denoted by the actual output string.

The released library uses Lean 4.34.0. Binary32/64 interoperation connects to Lean’s logical `Float32` and `Float` models [23]. The bridges cover integer constructors, addition and subtraction on finite inputs, and square root. Conversion to Lean’s packed models canonicalizes NaN signs and payloads while preserving numeric bits, including the sign of zero; the square-root theorem accounts for this canonicalization. For example, the addition bridge transports a left-associated native sum into certified software arithmetic:

```
import FloatLib
open FloatLib.Floats.ExecFloat.Binary

theorem import_sum3 (x y z : Float32)
  (hx : x.isFinite = true) (hy : y.isFinite = true)
  (hz : z.isFinite = true) (hxy : (x + y).isFinite = true) :
  ofFloat32 ((x + y) + z) =
    (ofFloat32 x + ofFloat32 y) + ofFloat32 z := by
  rw [ofFloat32_add_of_isFinite (x + y) z hxy hz,
    ofFloat32_add_of_isFinite x y hx hy]
  rfl
```

Only the inputs and first sum must be finite; the final sum may overflow. The equation uses Lean’s logical native model and preserves the two rounding steps.

3.7 Compilation and runtime assumptions

Lean erases proof fields when compiling arithmetic. Proved compiler simplification equations replace selected reference primitives with faster equal functions, provided the equation is imported before the caller is compiled. Closed descriptors and direct entry points expose representation and selection choices to inlining.

These proofs establish equations between Lean functions, including the logical models of native operations; they do not verify external machine instructions. Compiled execution relies on Lean’s compiler and runtime. The benchmarked `FloatLib` paths use proved software kernels. Separate `NativeFPU.Unchecked` adapters use host arithmetic without the same equality certificate. The native-C timings in Section 5 provide a hardware baseline.

4 Format-specific arithmetic

Each family connects its representation and reference operations to the execution certificate, while the numerical guarantees in Table 2 depend on that family’s representation and rounding rules. The posit proofs justify rounding at encoding boundaries and show how exact updates compose within a finite quire.

4.1 Posit rounding thresholds

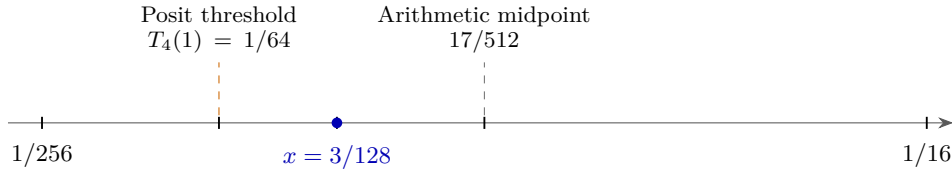
An n -bit posit uses a variable-length regime, up to two exponent bits, and the remaining fraction bits, concentrating precision near one. It has one zero and one exceptional value, NaR (“not a real”). The 2022 standard fixes the exponent convention and specifies rounding using the format one bit wider [34].

Let $D_n(u)$ denote the rational value of a nonnegative n -bit code u . For adjacent interior codes $u, u + 1$, the rounding threshold is

$$T_n(u) = D_{n+1}(2u + 1). \quad (6)$$

■ **Table 2** Numerical families in FloatLib and representative guarantees. Each family states the domain, range, and encoding conditions appropriate to its operations.

Family	Representation	Representative contract
Binary	Parameterized fields, bias, and exceptional encodings	Encoded arithmetic refinement; IEEE-style real rounding and exactness
Decimal	Coefficient and quantum; BID and DPD encodings	Datum-level arithmetic, rounding error, and status
Posit	Regime, two exponent bits; $n \geq 2$	Standard rounding; exact accumulation in a $16n$ -bit quire
P3109	Width, precision, signedness, and domain	Destination projection, saturation, and mixed-format operations
MX	32 lanes with a common E8M0 scale	Quantization at a selected scale; one-round exact dot product
Other codes	Fixed point, logarithmic, codebook, and shared scale	Scale-aware arithmetic or finite-set quantization



■ **Figure 4** Two different rounding boundaries for adjacent four-bit posits. The values are placed on a linear real axis. The standard posit rule rounds x to $1/16$ because it lies above the appended-code threshold; absolute nearest-value rounding would choose $1/256$.

Appending a one to the lower code gives the boundary in the wider format. At equality, the retained even code wins. Zero is exact; nonzero values below the least positive value round to that value in magnitude, and values above the greatest finite value saturate. Negative results use the corresponding symmetric rule.

For standard four-bit posits, codes 1 and 2 denote $1/256$ and $1/16$; five-bit code 3 gives the threshold $1/64$. Thus $x = 3/128$ rounds to code 2 although code 1 has smaller absolute error (Figure 4). Writing W_n for the n -bit posit words, we give $\Pi_n : \mathbb{R} \rightarrow W_n$ a separate real specification using the standard's encoding boundaries, and implement it through a search driven by comparisons with rational thresholds. This separates the finite code search from the target's comparison method.

► **Theorem 6** (Posit rounding from exact comparisons). *Let $n \geq 2$, $y \in \mathbb{R}$, and let $C : \mathbb{Q} \rightarrow \{<, =, >\}$ satisfy $C(t) = \text{cmp}(y, t)$ for every rational t . The signed comparator-based search returns $\Pi_n(y)$. In particular, the rational executable rounder returns $\Pi_n(q)$ on every rational input q .*

Proof. For an interior positive target, let c be the greatest code whose value does not exceed it. The bisection invariant is $\ell \leq c < h$ and $h - \ell \leq 2^f$, where f is the number of remaining steps. Starting from $[0, 2^{n-1})$, n steps suffice to isolate c . The comparator equation makes the executable and real searches take identical branches, finding the same lower code and agreeing at the threshold (6), including equality and its parity test. The zero, minpos, and saturation branches agree by the same equation. For a negative target, reversing comparisons with negated thresholds gives the magnitude comparator; whole-word negation restores the sign. Rational comparison supplies the equation for rational targets. ◀

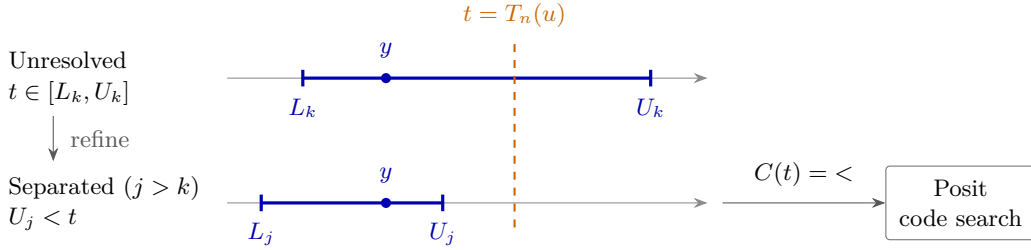
4.2 Exact comparison for elementary functions

For a rational radicand $a \geq 0$ and rational threshold $t \geq 0$, $t \leq \sqrt{a} \iff t^2 \leq a$ supplies an exact comparison. Negative thresholds lie below the root and are decided directly. The equivalence and its strict counterpart determine the code and threshold tests in Theorem 6, proving standard posit rounding of the exact root, including ties and extreme intervals.

Exponential and logarithm use rational enclosures $I_k = [L_k, U_k]$ of the real target y , at approximation degrees 2^k . Refinement continues until $U_k < t$ or $t < L_k$ decides the comparison with a rational threshold t (Figure 5). Containment makes each reported ordering sound; convergence of both endpoints to y ensures eventual separation when $y \neq t$. The equalities $\exp(0) = 1$ and $\log(1) = 0$ are handled directly. For rational $q \neq 0$, $\exp q$ is irrational, as is $\log q$ for $q > 0$, $q \neq 1$, excluding equality with rational thresholds in the remaining cases. The comparator supplies both code-value queries and the final threshold test in Theorem 6: for an ordinary word v denoting q ,

$$\exp_n(v) = \Pi_n(\exp q), \quad \log_n(v) = \Pi_n(\log q) \quad (q > 0).$$

Rational powers combine logarithm enclosures with exact integer-power comparisons; the latter resolve inconclusive enclosures and decide equality. Roots, trigonometric functions, and hyperbolic functions have corresponding domain-specific comparison proofs.



■ **Figure 5** Enclosure comparison at an interior posit threshold $t = T_n(u)$ from (6). Both rational intervals contain y . The first test is unresolved; the later bound $U_j < t$ certifies $y < t$ for the code search. The stages are schematic; neither nesting nor a convergence rate is assumed.

The optional binary elementary-function kernels compute deterministic approximations; their equality certificates do not establish correct rounding of the real function. Separate rational exponential and logarithm enclosures provide proved containment bounds. Appendix B proves separation and describes enclosure reuse and logarithmic comparison for large exponential arguments.

4.3 Exact quire accumulation

The standard quire for an n -bit posit is a signed $16n$ -bit accumulator with unit $\Delta_n = 2^{16-8n}$. Every product of two ordinary n -bit posits is an integer multiple of this unit, so products and partial sums remain exact while the accumulator stays in range. Conversion to a posit rounds the final sum. Write $\text{val}_n(v)$ and $Q(A)$ for the rational values of an ordinary posit and quire, and let $\text{foldMulAdd}(0, \mathcal{P})$ fold the model's exact product update from zero over operand pairs $\mathcal{P}$.

► **Theorem 7** (Exact quire accumulation). *For a standard posit width $n \geq 2$, let $\mathcal{P} = [(a_0, b_0), \dots, (a_{k-1}, b_{k-1})]$ contain no NaR operands. If $k < 2^{31}$, then the fold returns an ordinary quire and*

$$Q(\text{foldMulAdd}(0, \mathcal{P})) = \sum_{i < k} \text{val}_n(a_i) \text{val}_n(b_i).$$

Proof. The largest posit magnitude is $2^{4(n-2)}$, so a product has magnitude at most 2^{8n-16} , or $B_n = 2^{16n-32}$ in quire units. We induct on j , maintaining exactness and $|A_j| \leq jB_n$ for the quire coefficient. The base case is $A_0 = 0$. If the next product has coefficient C_j , then $|A_j + C_j| \leq |A_j| + |C_j| \leq (j+1)B_n$. For every prefix $j+1 \leq k < 2^{31}$, this bound is strictly below 2^{16n-1} . The next coefficient is therefore representable and distinct from the reserved NaR word, so the one-step arithmetic theorem preserves exactness and completes the induction. ◀

The bound is uniform in n because quire capacity and product bounds grow together; it holds in every ordering without relying on cancellation. Longer sums can succeed when their actual partial sums remain in range.

The configured update decodes operands through their codec and calls the model update. The codec laws transfer the fold result to `toRat?_fold1_qMulAdd_zero`, whose successful rational decoding also excludes NaR. For the resulting quire A , the model conversion theorem and codec round trip give

$$\text{toModel}(\text{qToP}(A)) = \text{roundRat}_n \left(\sum_{i < k} \text{val}_n(a_i) \text{val}_n(b_i) \right).$$

By Theorem 6, this is one standard posit rounding of the exact dot product.

4.4 Binary, decimal, and scaled formats

Low-precision binary. Small binary formats reuse field descriptors with different exceptional encodings [32, 33]. E5M2 and bfloat16 use IEEE exceptional conventions; E4M3FN has NaN patterns but no infinity. FNUZ reserves the negative-zero word for its unique NaN and changes the bias. MX’s FP4 and FP6 element formats have neither NaN nor infinity. The reference arithmetic applies the policy when packing results. For encodings in which every word is finite, such as FP4 and FP6, directed add, subtract, and multiply bounds require the exact result to lie within the finite range; these theorems do not cover the NaN-bearing FN and FNUZ policies.

A binary operation on byte-sized values admits a complete table under the same execution certificate. An eight-bit FMA table would require 2^{24} entries; the released FMA implementation uses the exact baseline.

Decimal arithmetic. Decimal datums retain the quantum exponent q as well as the value $(-1)^s c 10^q$: 1.50 and 1.5 are distinct, so BID and DPD encode both coefficient and quantum exponent [19]. FloatLib supports decimal32, decimal64, decimal128, and parameterized layouts with $3d+1$ digits, positive exponent-continuation width, and arbitrary integer bias. Arithmetic contracts preserve datum validity and status; nearest-even and nearest-away error bounds are half the selected decimal grid unit for nonoverflowing results. Division requires a nonzero divisor; square root requires a nonnegative finite input and retains the no-overflow premise for general descriptors.

P3109. P3109 operations follow working-group report 4.0.3 [20], with independent source and destination descriptors. They evaluate finite inputs exactly, then project into the destination under explicit rounding and saturation choices. Deterministic modes have direction and error results before saturation; the full projection theorem includes saturation. Stochastic modes take random bits explicitly and establish grid membership, which alone does not imply unbiasedness.

Microscaling. MX couples 32 lanes through a shared E8M0 scale [33]. At a selected finite scale, saturating conversion minimizes each lane’s error among finite encodings; the contract does not assert a global optimum over scales. For finite elements and scales, we choose exact product accumulation followed by one binary32 rounding for the MX dot product.

Fixed point and other encodings. Fixed-point, logarithmic, codebook, and generic shared-scale values use the representation interface of Section 2.1. Multiplying fixed-point codes at scales d_1, d_2 produces scale $d_1 + d_2$; codebook quantization minimizes distance over the table’s finite entries when at least one exists. Each family states its own scale and range conditions.

5 Evaluation

On the fixed scalar workload, FloatLib has lower medians than extracted Flocq in all 54 matched binary cases and than Universal in 34 of 65 matched posit cases, including all five operations at 64 bits and all twenty cases at widths 512–4096. The 31 slower cases include all twenty at widths 5–8. All 84 binary medians exceed MPFR’s. Separate Lean comparisons measure P3109 arithmetic and scalar conversions (Section 5.1); independent numerical checks follow.

Formats and execution. We time addition, subtraction, multiplication, division, square root, and FMA at binary widths 4, 5, 6, 7, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096. Widths 4–8 use E2M1, E2M2, E3M2, E3M3, and E4M3: E counts exponent bits and M stored fraction bits. Widths 16–128 use IEEE interchange layouts; larger widths use 19 exponent bits and $w - 20$ fraction bits, hence significand precision $P = w - 19$ including the leading bit. Posits add widths 2 and 3, with $es = 2$ throughout. Their precision varies with magnitude: a 32-bit posit can have 28 significand bits, against binary32’s 24. Equal encoded width does not imply equal accuracy.

Binary32/64 call proved word kernels directly; other binary widths and all posits use public throughput-policy dispatch. For binary addition, `backend-regimes.csv` records the paths described in Section 3: exhaustive tables at 4–8 bits, word kernels at 16–64, an internal fixed-limb kernel at 128, and the exact baseline at 256–4096. Wide rows use the default carrier; the optional limb-array carrier was not measured.

Workload and measurement. Each operation uses sixteen tuples rounded before timing (Appendix A.4). A result fingerprint selects the next tuple, creating a dependency without accumulating arithmetic results. Timing includes fixture selection, arithmetic, representation handling, and result observation; construction, compilation, and proof checking are excluded. Sources, trials, and environment details accompany the evaluation data.

Measurements use an Intel Xeon Platinum 8488C with affinity to one logical CPU; affinity does not reserve the physical core. C adapters use `-O3 -march=native` without fast-math; SoftFloat uses `-O2`. Each cell has nine fresh-process trials, with 256 warmup iterations for plotted implementations. A pilot targets 200 ms; accepted measurements exceed 50 ms. The record contains 448 cells and 4,032 observations. We report medians of elapsed time per

iteration and linearly interpolated empirical 5th–95th percentiles across trials, not confidence intervals.

Reference configurations. MPFR 4.2.0 rounds exact rational fixtures nearest-even [14], with $emin = 2 - B - (P - 1)$ and $emax = B + 1$ for significand precision P and exponent bias B . Every conversion and arithmetic operation passes its ternary rounding result to `mpfr_subnormalize` to match gradual underflow [29]. SoftFloat revision `a0c6494`, with the 8086–SSE specialization, and native C cover binary32/64 [17]. Native C checks representations, rounding mode, and fixture outputs against MPFR. Binary fingerprints and fixture traces agree on an untimed 256-step prefix; independently calibrated timed loops can differ in length. Native C supplies a loop baseline, not instruction latency.

Universal revision `26e69f5` receives the same encoded posit inputs and checks all sixteen fixture results [31]. Of 78 operation/width checks, 71 pass; the seven failures are square roots at widths 64–4096. The six passing square-root configurations use host binary64 and are also excluded, leaving 65 software comparisons. Flocq 4.2.2 supplies 54 binary comparisons at widths 16–4096 using its verified operators extracted to OCaml/Zarith [6], with matched precisions and fixtures (Appendix A.4).

Arithmetic performance. Figure 6 shows absolute costs across widths. Binary32 addition takes 138.6 ns in FloatLib, against 34.1 ns in MPFR and 22.9 ns in SoftFloat; all six binary32 medians exceed both references. At binary64, FloatLib addition rises to 312.8 ns and division from 123.7 to 1,447.3 ns. The overhead varies by operation and width.

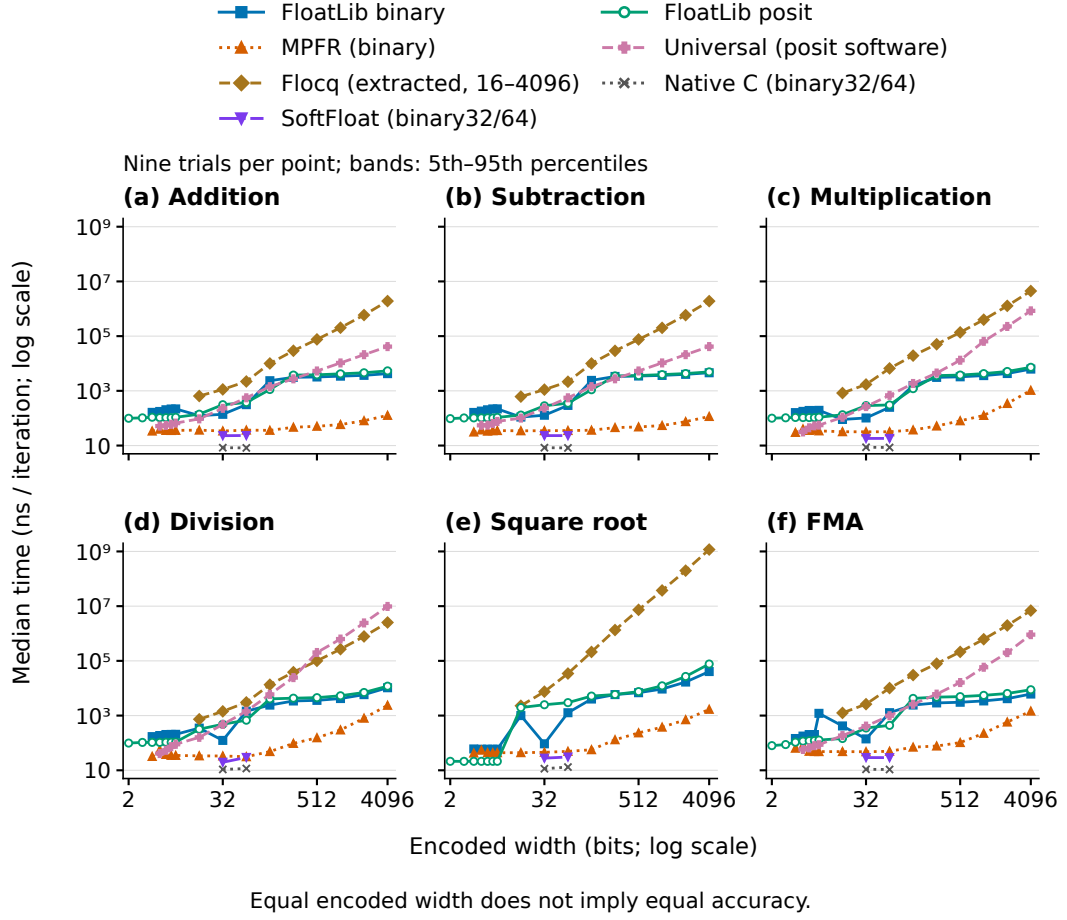
Figure 7 compares costs within each family. Posit multiplication takes 288.9 versus 266.5 ns at 32 bits, 305.1 versus 684.9 ns at 64 bits (Universal/FloatLib 2.25), and 7.25 versus 839.63 μ s at 4096 bits. The accompanying data give all 203 matched ratios and trials. Fixed fixtures and selected paths do not characterize arbitrary cancellation, exponent gaps, exceptional inputs, or application performance, nor isolate allocation, representation, and arithmetic costs.

5.1 Comparisons with Lean libraries

Separate experiments compare P3109 arithmetic with FLoPS and scalar low-precision conversions with TensorLib [10, 38]. On a Xeon Platinum 8275CL with affinity to one logical CPU, each case has nine paired fresh-process trials in alternating order, 256 warmup iterations, and sixteen shared fixtures. Results select subsequent inputs; timing includes lookup, the operation, encoded output, and checksums, excluding input construction, startup, and file output. The libraries use their respective supported toolchains; the ratios include compiler differences.

P3109 arithmetic. FloatLib has lower medians in nine of twelve cases (Table 3); FLoPS is faster for four-bit addition, multiplication, and FMA. FLoPS division costs 1.32–1.46 times FloatLib’s median. Eight-bit FMA differences are small: the empirical 5th–95th percentile intervals overlap for **Binary8p3se**. Both execute software arithmetic: FloatLib uses exact rationals, FLoPS dyadics and quotient-based rounding. These timings do not isolate representation costs.

The formats are **Binary4p2sf**, **Binary8p4se**, and **Binary8p3se**, with nearest-even rounding and no saturation. Addition, multiplication, and division agree on every ordered encoded pair, including exceptional values. FMA agrees on every four-bit triple and, for each eight-bit format, every encoded pair (x, y) with third operand code $(17x + 29y + 43) \bmod 256$. Together these checks compare 529,152 results with no bit differences; all 192 timing fixtures also agree.

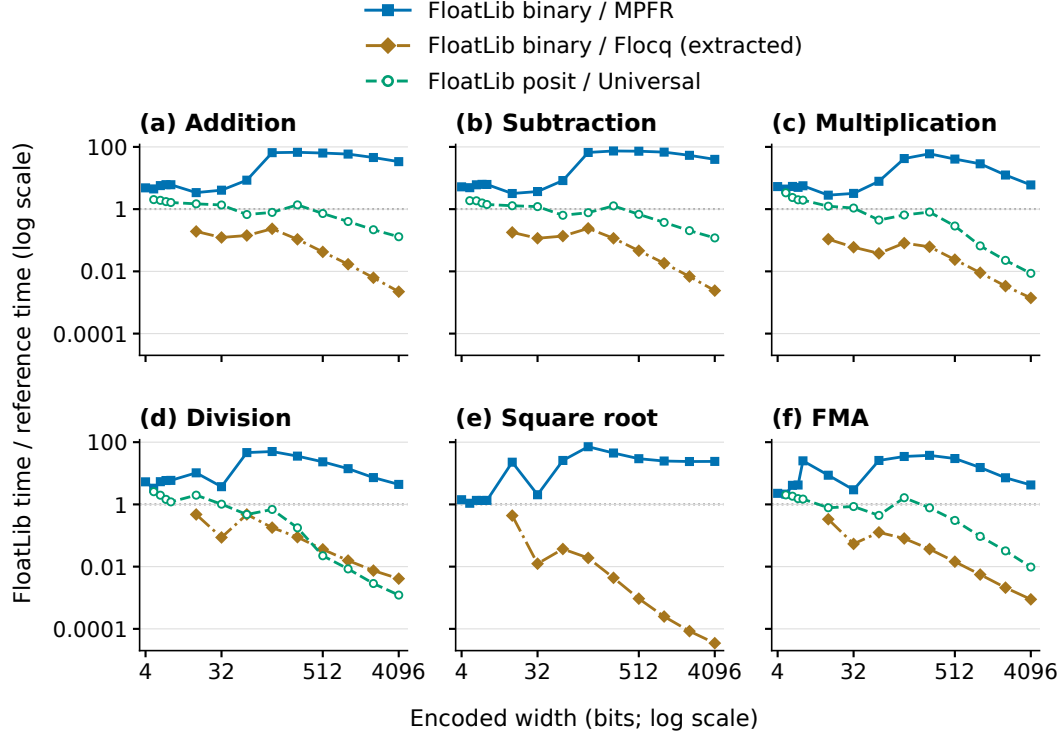


■ **Figure 6** Absolute scalar costs across encoded widths. Markers are medians of nine trials; bands show empirical 5th–95th percentiles. Both axes are logarithmic. Timing includes input selection and result observation. MPFR matches binary precision and range; extracted Flocq covers matching precisions at widths 16–4096. Universal uses the same posit words and $es = 2$, excluding host-assisted square roots. Equal width does not imply equal accuracy.

■ **Table 3** P3109 arithmetic: median μs per dependent-loop iteration across nine trials on a Xeon 8275CL. Inputs and nearest-even, nonsaturating policies match; the libraries use their respective Lean toolchains.

Format	Library	Add	Multiply	Divide	FMA
Binary4p2sf	FloatLib	4.37	4.47	4.32	5.86
Binary4p2sf	FLoPS	4.29	4.13	5.71	5.16
Binary8p4se	FloatLib	4.89	4.96	4.94	6.40
Binary8p4se	FLoPS	5.57	5.42	7.24	6.66
Binary8p3se	FloatLib	4.79	4.99	4.91	6.31
Binary8p3se	FLoPS	5.43	5.29	7.00	6.40

Low-precision conversion. FloatLib/TensorLib median ratios range from 5.9 to 13.0 (Table 4). All sixteen timed inputs are normal and exactly representable; subnormal and exceptional cases are unmeasured. TensorLib specializes normal conversions using bit-field operations;



Ratios of medians; values below 1 favour FloatLib.

■ **Figure 7** Relative costs within each family: FloatLib binary/MPFR, FloatLib binary/extracted Flocq, and FloatLib posit/Universal software. Each point divides two nine-trial medians; values below one favour FloatLib. Both axes are logarithmic. The 203 ratios comprise 84 MPFR, 54 Flocq, and 65 Universal comparisons; Flocq covers widths 16–4096 and Universal excludes square roots. Lines connect measured widths; ratios have no uncertainty bands.

FloatLib uses shared format-descriptor casts. Tensor arithmetic uses host floats (Section 6).

■ **Table 4** Low-precision conversion: median ns per dependent-loop iteration across nine trials on a Xeon 8275CL. Encoding converts binary32 to the listed format; decoding converts back. The sixteen timed inputs are normal finite values.

Format	Encode		Decode	
	FloatLib	TensorLib	FloatLib	TensorLib
FP16	2201.0	265.3	2098.2	268.9
bfloat16	2219.9	252.6	1590.0	268.3
E4M3FN	3642.9	279.7	2187.5	272.9
E5M2	2248.1	267.9	2083.1	270.5

Conversions connect binary32 with FP16, bfloat16, E4M3FN, and E5M2. Decoding checks all destination encodings; encoding checks finite destination values, adjacent midpoints and binary32 neighbours, both signs, exponent and overflow boundaries, NaNs, and 8,192 seeded random words per format. Both libraries match an independent nearest-even reference in all 952,612 cases, comparing non-NaN bits and NaN class. The 2,964 E4M3FN NaN-sign differences occur on overflow and infinity: FloatLib returns `7f`, TensorLib `ff`. Status flags and NaN payloads are not compared.

5.2 Independent numerical checks

Table 5 distinguishes counts and comparison relations. The TestFloat adapter calls logical `Model.*WithStatus` operations, not every timed backend. It checks non-NaN words and five flags; NaNs agree by quiet/signalling class, ignoring payload and sign [18]. Arithmetic and conversion cases use four rounding directions and tininess after rounding, excluding binary80, nearest-away, round-to-odd, and tininess-before-rounding. FMA accounts for 98,131,968 cases, or 95.78% of the TestFloat total.

■ **Table 5** Independent numerical comparisons. Counts include repeated evaluations; zero denotes agreement under the stated relation. Combined SoftPosit counts include the initial comparison. Decimal evidence consists only of summaries.

Reference	Tested property	Evaluations	Differences
TestFloat	Binary model words and five flags	102,454,320	0
MPFR	Primitive rounding; exact reductions	400 + 672	0
MPFR, binary16	Two fixed-first-operand rows	131,072	0
IBM FPgen	Selected binary32 results and flags	81,513	0
Z3 QF_FP	Result bits; NaN class	27,492	0
Decimal	Complete datum and five flags	13,980	0 reported
references			
SoftPosit, initial	Public posit results/comparisons	23,718,760	7,562; 5 inputs
SoftPosit, combined	Public posit results/comparisons	32,893,800	7,564; 7 inputs

MPFR correctness checks use version 4.1.0. The binary16 rows enumerate second operands for negative-zero addition and signalling-NaN multiplication, not all operand pairs. IBM admits 81,513 of 130,471 records after excluding decimal, missing-result, trap, and incompatible NaN/underflow cases. Z3 compares 25,168 results by bits and 2,324 NaNs by class. Separate ONNX and P3109 table checks concern decoding only. Counts and whole-suite runtimes supply no arithmetic speed ratio.

Decimal summaries report 10,425 arithmetic comparisons, 2,775 square roots bounded by outward MPFR intervals and rounded independently with libmpdec, and 780 exact or special-value cases: 13,980 total. They observe sign, coefficient, quantum exponent, special values, and five flags, distinguishing cohorts such as 1.50 and 1.5. Multiple NaNs follow the documented first-NaN choice; unspecified oracle-generated NaN signs are normalized. The released artifact includes summary reports for these comparisons, but not the raw vectors, runners, or tested source revision needed to replay them.

SoftPosit discrepancies. SoftPosit revision `17d5628185b3` supplies ten arithmetic and comparison operations. Initial checks exhaust word tuples at widths 2–8 and sample widths 16 and 32. Sampling widths 9–15 adds 9,175,040 evaluations and two differences. Excluding

repeated 16/32-bit cells gives 32,893,800 evaluations and 7,564 differences across seven distinct input tuples; 7,560 occurrences repeat three 32-bit inputs.

Four distinct inputs involve FMA. At width 14, $a = 09f4$, $b = 1f60$, and $c = 0001$ decode to $253/2048$, 2^{19} , and 2^{-48} . Thus $ab + c = 64768 + 2^{-48}$. Adjacent words `1efe` and `1eff` represent 64512 and 65024, with appended-bit rounding threshold 64768, also their midpoint here [34]. Their errors are $256 + 2^{-48}$ and $256 - 2^{-48}$: FloatLib correctly chooses the odd upper word; SoftPosit chooses the even lower word. Its generic FMA records discarded bits, then overwrites that sticky information at final rounding, turning a non-tie into a tie. This illustrates the obligation in Section 3.3: the discarded suffix’s nonzeroness must survive until rounding.

The other three inputs concern generic 32-bit addition, multiplication, and division. Packing has one exponent bit left, so exponent value two must contribute its upper bit, $2 \gg 1 = 1$. SoftPosit instead shifts by $28 - 29 = -1$, undefined in C/C++. Exact decoding and the standard’s thresholds select FloatLib’s results; SoftPosit’s named `p32` operations agree. On all seven inputs, public FloatLib operations agree with its exact specification. Recorded source probes confirm the two C diagnoses on these inputs, without proving a repaired implementation correct.

6 Related work

Shared numerical foundations. Flocq is our principal mathematical precedent. Boldo and Melquiond’s radix-and-exponent abstraction develops representability, rounding, and error results once for several numerical formats [4, 5]. We follow it and its separation of generic rounding from concrete arithmetic. Flocq 4.2.2 also provides verified executable binary operators, including exceptional values, bit encodings, the four arithmetic operations, square root, and FMA, parameterized by precision and exponent range [6]. FloatLib adds representation and quantization interfaces without a radix assumption (Section 2.1). Its alternative software kernels share an encoded-result specification, so numerical proofs remain valid when storage or algorithms change.

FloatSpec brings Flocq-style theory, IEEE models, and Hoare-style specifications to Lean [2]. Its `Calc.Operations.Fplus` aligns integer significands and adds them; `F2R_plus` proves that the real value is the exact sum. The rounded IEEE interfaces, including `Bplus`, `Bmult`, `Bdiv`, and `Bsqrt`, are noncomputable definitions in the cited version. FloatSpec also proves bridges between its binary64 operations and Lean’s logical `Float.Model1`; these connect specifications without verifying native machine instructions. Timing the executable `Fplus` in place of a rounded operation would omit the destination rounding and packing measured for FloatLib. FloatLib carries the rounded encoded specification through executable backend selection.

P3109 semantics, algorithms, and execution. FLoPS, by Chang, Park, Lim, and Nagarakatte, is the closest related Lean library for P3109 arithmetic [9, 10]. It connects bits to algebraic values and extended-real semantics with NaNs, with explicit rounding, saturation, and finite-domain policies. Its executable artifact proves refinement for addition, multiplication, division, and FMA: `add_refines`, for example, equates the decoded result with projection of the exact semantic sum [10]. FloatLib likewise connects executable arithmetic to numerical semantics, and provides same-format and mixed-format P3109 operations through its common execution interfaces.

FLoPS also studies what familiar numerical algorithms retain when the format changes. Its `FastTwoSum` analysis states the conditions under which error-free decomposition survives

saturation. Its `ExtractScalar` analysis shows that a magnitude bound on the extracted leading part can fail at precision one, where tie behavior differs from the usual even-significant rule [9]. Its stochastic-rounding theorems give finite-randomness bias formulas and root-mean-square bounds for sums of rounding errors under its sampling assumptions [10]. FloatLib’s explicit-random-bit refinement and grid-membership results establish the implemented choice and its admissible outputs; they do not supply that probabilistic analysis.

The specifications follow different working-group reports: FLoPS uses 3.2.1 and FloatLib uses 4.0.3 [9, 20]. We compare their common nearest-even, nonsaturating operations on matching formats. Section 5.1 reports the numerical checks and matched timings.

Native arithmetic and tensor storage. Primitive Floats exposes hardware binary64 arithmetic in Coq through axioms connecting primitive operations to their logical specification [3]. Martin-Dorel, Melquiond, and Roux connect these primitives to Flocq and use them in CoqInterval and ValidSDP [25]. FloatLib’s opt-in `NativeFPU.Unchecked` path similarly uses host binary32/64 operations, without the software-refinement certificate carried by our certified kernels.

TensorLib addresses the needs of tensor programs: dtypes, byte storage, shapes, strides, indexing, and broadcasting [38]. Its low-precision scalar arithmetic commonly decodes an FP8, FP16, or bfloat16 value into `Float32`, performs the native operation, and encodes the result; binary32/64 use Lean’s native floating-point types directly. FloatLib’s certified execution instead relates software kernels to their packed specifications. The scalar-conversion comparison in Section 5.1 times normal-value integer bit operations, so it does not measure an FPU advantage.

From arithmetic to expression and program proofs. Gappa combines interval reasoning, rewriting, and forward error analysis to bound expressions containing exact and rounded operators, producing proof certificates checkable in Coq [11]. VCFloat2 automates roundoff analysis through reification and proof by reflection; its specifications accommodate user-defined operations and nonstandard formats, and its bounds can be combined with VST program proofs [1]. Fluctuat uses interval and affine-set abstractions to track real values, finite-precision values, and error sources through programs with loops. It detects unstable tests, but the error analysis in the cited work assumes matching real and finite-precision control flow [16]. These systems propagate local arithmetic facts through larger computations. FloatLib supplies such facts for its numerical families; it does not include an automatic whole-program roundoff analysis.

SymFPU implements SMT floating-point operations using bit-vector operations, with interfaces that admit concrete execution or symbolic encodings for bit-blasting [7]. SymFPU connects floating-point semantics to a solver’s bit vectors; FloatLib connects packed operations to Lean implementations and numerical theorems. Our Z3 comparisons test selected SMT floating-point cases.

For verified interval computation, the desired conclusion is containment. Melquiond develops verified software arithmetic for numerical proofs in Coq [26]; later work shows how sufficient enclosure properties can replace correctly rounded directed operations [25]. Faissole, Geneau de Lamarlière, and Melquiond verify an optimized exponential enclosure, including its tables and bit manipulations, and integrate it into CoqInterval [13]. Containment is a different contract from correct scalar rounding. FloatLib similarly distinguishes its proved rational `exp/log` enclosures from executable binary transcendental approximations. The interval studies’ whole-proof timings and our scalar-operation timings measure different uses of verified arithmetic.

Connections within Lean. FloatLib connects to Lean’s logical `Float` and `Float32` models [23].

The bridges cover integer constructors, finite-input addition and subtraction, and square root with NaN canonicalization (Section 3.6). They transfer numerical facts through the logical models while retaining the assumptions of native execution. DsLean translates external domain-specific representations into typed Lean terms. Its Gappa case study reconstructs proofs for exact real interval arithmetic [36]. Supporting rounded operations in that translation also requires the corresponding arithmetic theorems.

7 Conclusion

FloatLib provides arbitrary-precision floating-point arithmetic in Lean with user-defined formats and rounding rules. Its certified execution interface connects optimized software kernels to complete encoded specifications, with numerical theorems establishing rounding, error bounds, and exactness under their stated assumptions. Shared rounding invariants and quotient checks allow implementations to avoid unnecessary computation without changing their results. Proofs about a numerical program can therefore use the same arithmetic specification as its implementation is optimized.

The library, proofs, benchmarks, evaluation data, and guide are available as open source at <https://github.com/lean-dojo/FloatLib>.

References

- 1 Andrew W. Appel and Ariel E. Kellison. VCFlo2: Floating-point error analysis in Coq. In *Proceedings of the 13th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2024)*, pages 14–29, New York, NY, January 2024. ACM. doi:10.1145/3636501.3636953.
- 2 Beneficial AI Foundation. FloatSpec. Source repository, version 0.7.0, 2026. Source revision 158263e. URL: <https://github.com/Beneficial-AI-Foundation/FloatSpec/tree/158263e983ec3925e02b10f5b312498bf414e0f1>.
- 3 Guillaume Bertholon, Érik Martin-Dorel, and Pierre Roux. Primitive floats in Coq. In John Harrison, John O’Leary, and Andrew Tolmach, editors, *10th International Conference on Interactive Theorem Proving (ITP 2019)*, volume 141 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 7:1–7:20, Dagstuhl, Germany, 2019. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ITP.2019.7.
- 4 Sylvie Boldo and Guillaume Melquiond. Flocq: A unified library for proving floating-point algorithms in Coq. In *2011 IEEE 20th Symposium on Computer Arithmetic (ARITH 2011)*, pages 243–252. IEEE, July 2011. doi:10.1109/ARITH.2011.40.
- 5 Sylvie Boldo and Guillaume Melquiond. *Computer Arithmetic and Formal Proofs: Verifying Floating-Point Algorithms with the Coq System*. ISTE Press and Elsevier, 2017. URL: <https://inria.hal.science/hal-01632617>.
- 6 Sylvie Boldo and Guillaume Melquiond. Flocq. Software, version 4.2.2, February 2026. URL: <https://flocq.gitlabpages.inria.fr/releases/flocq-4.2.2.tar.gz>.
- 7 Martin Brain, Florian Schanda, and Youcheng Sun. Building better bit-blasting for floating-point problems. In Tomáš Vojnar and Lijun Zhang, editors, *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2019), Part I*, volume 11427 of *Lecture Notes in Computer Science*, pages 79–98, Cham, 2019. Springer. doi:10.1007/978-3-030-17462-0_5.
- 8 Erin Carson and Nicholas J. Higham. Accelerating the solution of linear systems by iterative refinement in three precisions. *SIAM Journal on Scientific Computing*, 40(2):A817–A847, 2018. doi:10.1137/17M1140819.
- 9 Tung-Che Chang, Sehyeok Park, Jay P. Lim, and Santosh Nagarakatte. FLoPS: Semantics, operations, and properties of P3109 floating-point representations in Lean. Technical Report DCS-

- TR-762, Rutgers University, Department of Computer Science, May 2026. arXiv:2602.15965v3, revised 17 May 2026. URL: <https://arxiv.org/abs/2602.15965v3>, arXiv:2602.15965v3.
- 10 Tung-Che Chang, Sehyeok Park, Jay P. Lim, and Santosh Nagarakatte. Semantics, operations, and properties of P3109 floating-point representations in Lean. Source repository, 2026. FLoPS executable artifact, source revision 4ead906. URL: <https://github.com/rutgers-apl/FLoPS/tree/4ead906d8bceefd400620dcc52ec3245f84669e9>.
 - 11 Marc Daumas and Guillaume Melquiond. Certification of bounds on expressions involving rounded operators. *ACM Transactions on Mathematical Software*, 37(1):1–20, January 2010. doi:10.1145/1644001.1644003.
 - 12 Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction (CADE 28)*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635, Cham, 2021. Springer. doi:10.1007/978-3-030-79876-5_37.
 - 13 Florian Faissole, Paul Geneau de Lamarlière, and Guillaume Melquiond. End-to-end formal verification of a fast and accurate floating-point approximation. In Yves Bertot, Temur Kutsia, and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:18, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ITP.2024.14.
 - 14 Laurent Fousse, Guillaume Hanrot, Vincent Lefèvre, Patrick Pélissier, and Paul Zimmermann. MPFR: A multiple-precision binary floating-point library with correct rounding. *ACM Transactions on Mathematical Software*, 33(2):13, June 2007. doi:10.1145/1236463.1236468.
 - 15 GNU MP. Manual, version 6.3.0. URL: <https://gmp1ib.org/manual/>.
 - 16 Eric Goubault and Sylvie Putot. Static analysis of finite precision computations. In Ranjit Jhala and David Schmidt, editors, *Verification, Model Checking, and Abstract Interpretation (VMCAI 2011)*, volume 6538 of *Lecture Notes in Computer Science*, pages 232–247, Berlin, Heidelberg, 2011. Springer. doi:10.1007/978-3-642-18275-4_17.
 - 17 John R. Hauser. Berkeley SoftFloat, release 3e. Software, January 2018. URL: <https://www.jhauser.us/arithmetic/SoftFloat.html>.
 - 18 John R. Hauser. Berkeley TestFloat, release 3e. Software, January 2018. URL: <https://www.jhauser.us/arithmetic/TestFloat.html>.
 - 19 IEEE. IEEE standard for floating-point arithmetic. IEEE Std 754-2019 (revision of IEEE Std 754-2008), 2019. doi:10.1109/IEEESTD.2019.8766229.
 - 20 IEEE SA P3109 Working Group. IEEE working group P3109 interim report on arithmetic formats for machine learning. Version 4.0.3, September 2026. Dated 1 September 2026; unapproved working-group report. URL: <https://github.com/P3109/Public/blob/34f5964d9bb2382b2665d15467fc3517b990b308/IEEE%20P3109%20Interim%20Report.pdf>.
 - 21 Eliska Kloberdanz, Kyle G. Kloberdanz, and Wei Le. DeepStability: A study of unstable numerical methods and their solutions in deep learning. In *Proceedings of the 44th International Conference on Software Engineering (ICSE 2022)*, pages 586–597, New York, NY, 2022. ACM. doi:10.1145/3510003.3510095.
 - 22 Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, Reading, MA, USA, third edition, 1997. URL: <https://www-cs-faculty.stanford.edu/~knuth/taocp.html>.
 - 23 Lean FRO. Lean 4.34.0 floating-point logical models. Source code, tag v4.34.0, 2026. Revision 293d5d0. URL: <https://github.com/leanprover/lean4/tree/293d5d0c0c3f3dded4688b3ccd6a33939ac5102b/src/Init/Data/Float/Model>.
 - 24 Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level

- code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. doi:10.1126/science.abq1158.
- 25 Érik Martin-Dorel, Guillaume Melquiond, and Pierre Roux. Enabling floating-point arithmetic in the Coq proof assistant. *Journal of Automated Reasoning*, 67(4), 2023. Article 33. doi:10.1007/s10817-023-09679-x.
 - 26 Guillaume Melquiond. Floating-point arithmetic in the Coq system. *Information and Computation*, 216:14–23, July 2012. doi:10.1016/j.ic.2011.09.005.
 - 27 Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018. URL: <https://openreview.net/forum?id=r1gs9JgRZ>.
 - 28 Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. FP8 formats for deep learning. arXiv:2209.05433, 2022. URL: <https://arxiv.org/abs/2209.05433>.
 - 29 *GNU MPFR*, 2023. Manual, version 4.2.0. URL: <https://www.mpfr.org/mpfr-4.2.0/mpfr.html>.
 - 30 Jean-Michel Muller, Nicolas Brunie, Florent de Dinechin, Claude-Pierre Jeannerod, Mioara Joldes, Vincent Lefèvre, Guillaume Melquiond, Nathalie Revol, and Serge Torres. *Handbook of Floating-Point Arithmetic*. Birkhäuser, Cham, second edition, 2018. doi:10.1007/978-3-319-76526-6.
 - 31 E. Theodore L. Omtzigt and James Quinlan. Universal numbers library: Multi-format variable precision arithmetic library. *Journal of Open Source Software*, 8(83):5072, March 2023. doi:10.21105/joss.05072.
 - 32 ONNX. Float stored in 8 bits. ONNX technical documentation, 2023. URL: <https://onnx.ai/onnx/technical/float8.html>.
 - 33 Open Compute Project. OCP microscaling formats (MX) specification, version 1.0. OCP specification, September 2023. URL: <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>.
 - 34 Posit Working Group. Standard for posit arithmetic (2022). Posit standard, 2022. Dated 2 March 2022; sponsored by the National Supercomputing Centre (NSCC) Singapore. URL: https://posithub.org/docs/posit_standard-2.pdf.
 - 35 Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. arXiv:2504.21801v1, 2025. URL: <https://arxiv.org/abs/2504.21801v1>.
 - 36 Tate Rowney, Riyaz Ahuja, Jeremy Avigad, and Sean Welleck. DSLean: A framework for type-correct interoperability between Lean 4 and external DSLs, 2026. Preprint, arXiv:2602.18657v3, revised 30 July 2026. URL: <https://arxiv.org/abs/2602.18657v3>, arXiv:2602.18657.
 - 37 Pat H. Sterbenz. *Floating-Point Computation*. Prentice-Hall, Englewood Cliffs, NJ, 1974.
 - 38 TensorLib contributors. TensorLib. Source repository, 2026. Source revision 85e61c9. URL: <https://github.com/leanprover/TensorLib/tree/85e61c9eb3211c736ded72b37254e2d608bbdbeb>.
 - 39 The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, New York, NY, January 2020. ACM. doi:10.1145/3372885.3373824.

■ **Table 6** Execution proof stages and their Lean declarations. Names are relative to the namespace in each shaded heading.

Proof step	Declaration
<i>Shared invariants</i> FloatLib.Numerics	
Preserve rounding after jamming	roundShiftRightEven_shiftRightJam
Account for the subtraction borrow	shiftRightJam_mul_two_pow_sub LimbArray.subLoop_spec
<i>Wide-limb arithmetic</i> FloatLib.Floats.Formats.BinaryInterchange.Model.WideLimb	
Normalize and pack the result	round_eq_of_roundJammed roundNormal?_map_toModel
Combine alignment branches	alignAndRound?_refines
<i>Quotient validation and rounding</i> FloatLib.Numerics.FixedWord	
Check the quotient and remainder	CertifiedDivision.certificate_sound CertifiedDivision.certificate_complete
Prove the fallback and its capacity	RestoringQuotient.quotientSteps128_spec CertifiedDivision.checkedCandidate_sound
Round the exact quotient	CertifiedDivision.roundQuotient_toNat
<i>Encoded operations and real interpretation</i> FloatLib.Floats	
Certify complete subtraction	Formats.BinaryInterchange.Configured.Plan.wideLimbSub
Preserve the specification through selection	ExecFloat.Backend.selectCertified_run_eq_spec
Apply the real FMA theorem	Formats.BinaryInterchange.Model.toReal_fma_eq_roundAt

A Execution refinement invariants

The execution proof in Section 3 proceeds from integer and limb invariants through packing to complete encoded operations. Table 6 groups the Lean declarations by these proof stages. We retain the notation J_j for jamming and E_s for nearest-even integer rounding. The parameter p denotes stored fraction width, so normal significands have $p + 1$ bits.

A.1 Jamming, normalization, and packing

To prove the suffix invariant in Theorem 4, decompose, for $k > 0$,

$$x = 2^j(A2^k + B) + v, \quad 0 \leq B < 2^k, \quad 0 \leq v < 2^j.$$

If $v = 0$, jamming returns $A2^k + B$. Otherwise it returns $A2^k + (B \text{ OR } 1)$. Since $k > 0$, setting bit zero leaves the suffix below 2^k . In either case the quotient is A , and the suffix is zero exactly when $B = v = 0$. This proves the quotient and suffix identities together; taking the quotient modulo two also proves the positive-index bit identity. At the guard position $k = s - j - 1$, the condition $j + 2 \leq s$ makes this argument applicable. It is a uniform separation condition: for every $j \geq 1$, $x = 2^{j+1} + 1$ gives $J_j(x) = 3$ and $E_1(3) = 2 \neq E_{j+1}(x) = 1$. Particular inputs can still agree without separation; in particular, J_0 is the identity.

An operation must also preserve the exponent used for packing. For $x \geq 2^{j+1}$, the truncated quotient is at least two, and $\lfloor \log_2 J_j(x) \rfloor = \lfloor \log_2 x \rfloor - j$. Indeed, low-bit insertion

either leaves the quotient unchanged or increments an even quotient. The latter cannot cross a power-of-two boundary, since the integer immediately below that boundary is odd.

Write $L = \lfloor \log_2 x \rfloor$. The stronger bound $x \geq 2^{p+j+2}$ ensures $L - j \geq p + 2$, so the jammed rounder's shift is $(L - j) - p = (L - p) - j$. Its rounded significand therefore equals $m = E_{L-p}(x)$ by Theorem 4. Write w for exponent width and $h = 2^{w-1} - 1$ for conventional bias. For an IEEE-style descriptor eligible for the wide-limb backend, $1 + w + p > 128$ and $w \leq 32$. The unsigned argument `scale`, denoted σ , represents the unrounded magnitude $x 2^{\sigma-2(h+p-1)}$, with exponent subtraction in $\mathbb{Z}$. The jammed rounder retains this argument and adds j when computing the packing position: $(L - j) + j + \sigma = L + \sigma$. Write $\rho = L + \sigma$ for this common position. Their common significand satisfies $2^p \leq m \leq 2^{p+1}$, so the normalization carry is exactly $c = \mathbf{1}[m = 2^{p+1}]$. The normal rounder accepts exactly when

$$h + 2p - 1 \leq \rho, \quad \rho + c \leq 3h + 2p - 2. \quad (7)$$

The packed fields are $e = \rho + c - (h + 2p - 2)$ and $f = m \bmod 2^p$. The acceptance bounds give $1 \leq e \leq 2h = 2^w - 2$ and $0 \leq f < 2^p$. Since $w \leq 32$, conversion of e to `UInt32` is exact, and taking the low p limb bits computes f exactly. If $c = 0$, then $f = m - 2^p$. If $c = 1$, then $m = 2^{p+1}$ and $f = 0$: the increased exponent accounts for renormalization. These bounds and field equations identify the packed result. The natural-number theorem separately handles $j = 0$ with only $x \neq 0$, including left normalization when $L < p$. The limb refinement then gives the same model result for an array denoting $J_j(x)$ whenever the normal branch accepts. Declined results are handled by the complete operation.

A.2 Limb subtraction and cancellation

Let $a, b \geq 2^p$, $s_b \leq s_a$, $d = s_a - s_b$, and $\ell_a = \lfloor \log_2 a \rfloor$, $\ell_b = \lfloor \log_2 b \rfloor$. In the opposite-sign compressed branch, $d \geq 3$ and $\ell_b + 2 \leq \ell_a + d$. Put $j = d - 3$, $T = a2^d - b$, and $b = Q2^j + r$ with $0 \leq r < 2^j$. The guard supplies two distinct bounds:

$$\begin{aligned} T &\geq 2^{\ell_a+d-1} \geq 2^{p+j+2}, \\ Q &< 2^{\ell_a+2} \leq 4a, \quad Q + 1 \leq 8a. \end{aligned} \quad (8)$$

The first supplies the rounding separation. The second makes the subtraction of Q and the incoming borrow $\mathbf{1}[r \neq 0]$ nonnegative. It follows by dividing $b < 2^{\ell_a+d-1} = 2^{\ell_a+2}2^j$.

To justify that subtraction for any limb count, set $\beta = 2^{32}$. Zero-extend the shorter array. For digits $u_i, v_i < \beta$ and incoming borrow $b_i \in \{0, 1\}$, the loop computes

$$\begin{aligned} t_i &= u_i + \beta - v_i - b_i, \quad z_i = t_i \bmod \beta, \quad b_{i+1} = \mathbf{1}[t_i < \beta], \\ u_i + \beta b_{i+1} &= z_i + v_i + b_i. \end{aligned}$$

Here $0 \leq t_i < 2\beta < 2^{64}$, so the 64-bit intermediate is exact. The induction preserves nonnegativity of the unprocessed suffix: if $V + b_i \leq U$ and $U = u_i + \beta U'$, $V = v_i + \beta V'$, the displayed identity and $z_i < \beta$ imply $V' + b_{i+1} \leq U'$. After the last digit this forces the final borrow to vanish. Consequently, for arrays denoting U, V and borrow $b_0 \leq 1$, the result denotes $U - V - b_0$ whenever $V + b_0 \leq U$. The allocated length is the maximum of the two input lengths; neither equal lengths nor a fixed upper bound on U, V is required.

Apply this contract with $U = 8a$, $V = Q$, and $b_0 = \mathbf{1}[r \neq 0]$. The complemented suffix $2^j - r$ in the inexact case, as derived in Section 3.3, explains why the borrow precedes low-bit insertion. The resulting limb value is exactly $J_j(T)$. The positive minuend ensures a nonempty result array, which is the storage premise for inserting the low bit.

When the dominance guard fails, alignment forms $a2^d$ exactly and compares it with b before subtracting. This retains arbitrarily deep cancellation, including exact equality, which returns positive zero for opposite signs. For an eligible descriptor, the accepted-branch theorem permits arbitrary scales and signs and imposes only $a, b \geq 2^p$ on the magnitudes. Thus an exact FMA product $m_x m_y \geq 2^{2p} \geq 2^p$ meets the same contract even though it can exceed the stored significand width. Its limb multiplication and alignment introduce no intermediate rounding.

A.3 Quotient certification and repair

Put $M = 2^{128}$. The checker takes $n, d, q, r < M$, with $64 < p \leq 126$ and $s \in \{p, p+1\}$. Its soundness and completeness theorems give acceptance exactly when $qd + r = n2^s$ and $r < d$. They require neither normalized inputs nor the comparison-directed choice of s . The widening lemma uses $64 < s < 128$; in particular, $n2^s < 2^{128+s} \leq 2^{255} < M^2$.

If u, c denote the value and carry of the four-word sum, multiplication and addition refinement give $u + cM^2 = qd + r$. The soundness proof uses the checked conjunct $c = 0$ to recover exact addition. Nevertheless, the argument widths alone imply

$$qd + r \leq (M-1)^2 + (M-1) = M^2 - M < M^2. \quad (9)$$

The carry test is therefore redundant for these types, including rejected candidates; it does not exclude any representable Euclidean decomposition. The remainder bound implies $d > 0$, and uniqueness of Euclidean division identifies q and r .

Repair needs the stronger domain $2^p \leq n, d < 2^{p+1}$, with $s = p$ when $d \leq n$ and $s = p+1$ otherwise. It starts from $q_0 = \mathbf{1}[d \leq n]$, $r_0 = n - q_0 d$. Normalization gives $n < 2d$, so $r_0 < d$. After i steps, the loop invariant is

$$q_i d + r_i = n2^i, \quad r_i < d, \quad (q_i + 1)2^{s-i} \leq M \quad (0 \leq i \leq s). \quad (10)$$

Initially $(q_0 + 1)2^s = 2^{p+1} \leq M$. A step sets $b = \mathbf{1}[2r_i \geq d]$, $q_{i+1} = 2q_i + b$, and $r_{i+1} = 2r_i - bd$. Because $d < 2^{127}$, doubling the remainder fits in 128 bits. If a step remains, the capacity invariant implies $2q_i + 1 < M$; the inequality $q_{i+1} + 1 \leq 2(q_i + 1)$ then preserves capacity for the remaining steps. At termination the pair satisfies the checker. The runtime selects this proved repair directly after rejection, without executing a second check.

The quotient-rounding theorem requires the strict increment bound $q + 1 < M$, together with $qd + r = N$, $r < d$, and $d < 2^{127}$, for arbitrary natural N . These bounds justify exact doubling of r and incrementing q . The normalized caller supplies them because its scaled quotient lies in $[2^p, 2^{p+1})$, giving $q + 1 \leq 2^{p+1} \leq 2^{127} < M$. The comparisons of $2r$ with d , including quotient parity at equality, then compute exact nearest-even rounding of N/d .

Encoded refinement. For an eligible implementation, let D, C be the codec's decoding and encoding maps, S the model specification, and K the complete binary operation. Combining the accepted-branch proof with the reference branch gives $D(K(x, y)) = S(Dx, Dy)$; hence $K(x, y) = C(D(K(x, y))) = C(S(Dx, Dy))$. This is equality of encoded results, including the specified zero signs and exceptional encodings. The reference baseline executes its supplied specification, so its refinement certificate is reflexive; selection preserves that same specification. Numerical correctness comes from the separate model theorem. In particular, the real FMA corollary uses an IEEE-style descriptor, finite operands, and a finite result, whereas the complete encoded equality has no operand restrictions.

A.4 Benchmark fixtures and Flocq comparison

The scalar experiment in Section 5 prepares sixteen tuples. For $0 \leq i < 16$, define

$$r_s(i) = \frac{((is + s + 1) \bmod 113) + 7}{((11i + s) \bmod 29) + 32}. \quad (11)$$

The first operand uses $r_{37}(i)$, negated when $i \bmod 5 = 0$; the second uses $r_{61}(i)$, negated when $i \bmod 3 = 0$. Square root uses positive $r_{43}(i)$; FMA's third operand reverses the first sequence. All inputs are rounded before timing.

Our Flocq 4.2.2 adapter extracts `BinarySingleNaN` arithmetic to OCaml/Zarith [6]. It matches significant precision, uses exponent bound 16384, and constructs each rational by rounding its numerator and denominator before division. At widths 16–4096, these integers are exactly representable, so both libraries receive the same inputs. Every nonzero fixture result is normal in both exponent ranges. Exact recomputation of all sixteen results per cell matches the recorded result and fixture-sequence checksums in all nine Flocq trials, as well as the FloatLib/MPFR agreement prefixes. These 54 comparisons are included in both performance figures. We omit widths 4–8 because the adapter can change their inputs: at precision two, for example, it constructs -1.5 from $-45/40$, whereas rounding that rational directly gives -1 .

B Exact comparison and quire invariants

Theorem 6 bounds the finite code search; comparing its real target with a rational boundary needs a separate termination argument.

B.1 Termination of enclosure comparison

For intervals $I_k = [L_k, U_k]$ with rational endpoints and a boundary $t \in \mathbb{Q}$, the comparison procedure finds $k_* = \min\{k \in \mathbb{N} : U_k < t \vee t < L_k\}$ and returns $<$ or $>$, respectively. The `Nat.find` search takes an existence proof, erased during compilation.

► **Lemma 8** (Soundness and termination of enclosure comparison). *Suppose $L_k \leq y \leq U_k$ for every k , both endpoint sequences converge to y , and $y \neq t$. Then k_* exists and the returned ordering is $\text{cmp}(y, t)$.*

Proof. If $y < t$, set $\varepsilon = (t - y)/2 > 0$. Convergence of the upper endpoints eventually gives $|U_k - y| < \varepsilon$, hence $U_k < y + \varepsilon < t$. If $t < y$, convergence of the lower endpoints similarly gives $t < L_k$. Thus one of the tests eventually succeeds. At the first successful test, containment proves the reported order: $y \leq U_{k_*} < t$ or $t < L_{k_*} \leq y$. ◀

Neither nested intervals nor a known convergence rate is required. Equality needs a separate decision: the intervals $[1 - 2^{-k}, 1 + 2^{-k}]$ converge to the boundary $t = 1$ but never separate it. Each interval admits targets on both sides of t , so a fixed refinement limit cannot decide equality.

For rational $q \neq 0$, irrationality of $\exp q$ excludes equality with rational boundaries; for $\log q$ we need $q > 0$ and $q \neq 1$. Together with endpoint convergence at degrees 2^k and direct comparison of $\exp 0 = 1$ and $\log 1 = 0$, this yields total comparators. Within `FloatLib.Numerics.Enclosure`, `Comparison.exists_separating` proves termination and `Comparison.compare_eq_real` proves soundness. The lemmas `exists_exp_separating` and `exists_log_separating` specialize termination to the exponential and logarithm. The resulting comparison equation supplies the hypothesis of Theorem 6.

B.2 Enclosure reuse

Rounding reuses Taylor enclosures across boundary queries for the same target. We prepare a finite array of lazy enclosures outside the boundary function, so the queries share evaluated entries. For every cache length c , `cacheIntervals_apply` proves $\text{get}(\text{cache}(I, c), k) = I_k$ for all k : entries with $k \geq c$ call the original generator, so the cache length does not limit refinement.

For a positive rational threshold t , $\exp q < t \iff q < \log t$. We reverse an exact comparison of $\log t$ with q to avoid constructing a potentially enormous exponential; nonpositive thresholds are decided immediately because $\exp q > 0$. The prepared comparator uses direct exponential enclosures for moderate arguments and the logarithmic route otherwise. Both satisfy the same comparison equation. The scalar timing experiment excludes these elementary-function implementations.

B.3 Quire accumulation from a partial sum

For a standard width $n \geq 2$, let an ordinary quire coefficient A denote $v = A\Delta_n$, with $\Delta_n = 2^{16-8n}$. Suppose $m, k \in \mathbb{N}$, $|A| \leq mB_n$, and $m + k < 2^{31}$, where $B_n = 2^{16n-32}$ and m counts the product bounds already consumed. For k remaining ordinary operand pairs with exact products $p_0, \dots, p_{k-1}$, the fold continuation represents $v + \sum_{i < k} p_i$. After $j \leq k$ updates, the induction invariant gives the exact value $v + \sum_{i < j} p_i$ and a coefficient A_j satisfying

$$|A_j| \leq (m + j)B_n < 2^{31}B_n = 2^{16n-1}.$$

The initial state satisfies this invariant by assumption. The triangle inequality advances the first bound, and $m + j \leq m + k < 2^{31}$ gives the second. Every state is ordinary, so the one-step theorem preserves exactness. The model theorem `toRat?_foldMulAdd` formalizes this induction; $A = v = m = 0$ recovers Theorem 7.

Ordinary coefficients satisfy $-2^{16n-1} < A < 2^{16n-1}$ because the most-negative two's-complement word is NaR. Maximum-magnitude products attain B_n quire units, so 2^{31} same-sign maximum products reach a boundary: the positive coefficient overflows, and the negative coefficient is the reserved NaR word. The update returns NaR in both cases. The strict bound covers every operand ordering, although cancellation can allow longer sums.

At the configured interface, `ExecFloat.Posit.Quire.toRat?_fold1_qMulAdd_zero` transfers the fold theorem through the operand codec. Its successful rational decoding identifies the exact product sum and excludes NaR. The model theorem `qToP_eq_roundRat` and codec round trip give one rounding of the sum; `RealRounding.round_ratCast` identifies it with Π_n .